

Examen de mi-semester SRCS Master 1 SAR

Jonathan Lejeune

Mars 2018

2 heures – **Tout document papier interdit**

Tout appareil électronique interdit

- Le barème est sur 21 points et est donné à titre indicatif. La note finale sera bornée à 20 points.
- Dans le code java demandé vous ne gérerez pas les imports.
- Il vous est conseillé de lire attentivement les exercices entièrement avant de composer.

Exercice 1 – Questions de cours (7 points)

Question 1 1 point

Dans le protocole RPC, qu'est ce que le portmapper ?

Question 2 1 point

À quoi sert un filtre XDR ?

Question 3 1 point

Qu'est-ce qu'un callback ? donnez un cas d'utilisation.

Question 4 1 point

À quoi sert le mot-clé `transient` ?

Question 5 1 point

Comment est géré le transfert d'un objet qui est à la fois `Remote` et à la fois `Serializable` ?

Question 6 1 point

Quel problème rencontre-t-on lorsqu'une classe décrivant un objet distant implante directement l'interface `Remote`, sans implanter une interface intermédiaire spécifiant les méthodes que l'objet doit offrir ?

Question 7 1 point

Que faut-il faire pour que le `rmiregistry` puisse utiliser les types définis dans votre programme ? Pourquoi ceci est-il nécessaire ?

Exercice 2 – Barrière de synchronisation en RMI (5 points)

Nous souhaitons développer un service RMI permettant de gérer des barrières de synchronisation entre plusieurs clients. Pour rappel, une barrière de synchronisation permet d'assurer que N processus aient tous passé un point spécifique de leur programme. Ainsi, dans notre cas, chaque client souhaitant se synchroniser sur une barrière de nom a devra attendre jusqu'à ce que N_a clients se soient synchronisés sur la barrière a . On dira que a est terminée lorsque que N_a clients se sont synchronisés dessus. Le service RMI doit donc offrir la possibilité à un client de :

- créer et d'initialiser une barrière en précisant son nom et sa valeur initiale. Si il existe déjà une barrière du même nom alors l'initialisation doit jeter une exception.
- de se synchroniser sur une barrière en spécifiant son nom. Une exception doit être jetée, si le nom spécifié ne concerne aucune barrière existante, ou alors si la barrière est terminée.

On devra assurer une isolation entre les différentes barrières, c'est à dire que les synchronisations sur une barrière x ne devront pas impacter une autre barrière y et vice-versa.

Question 1 5 points

Écrire les interfaces et classes Java permettant de résoudre ce problème.

Exercice 3 – Déploiement de services distants (9 points)

Nous souhaitons pouvoir déployer sur un ensemble d'hôtes des **services** invocables à distance.

Un **service** est un objet distant qui doit offrir les fonctionnalités suivantes :

- **getName** : renvoie une chaîne de caractères égale au nom du service. On considérera que ce nom identifie de manière unique le service dans le système.
- **invoke** : la méthode permettant d'invoquer le service. Elle prendra comme seul argument, un objet quelconque transmis par copie qui permettra de paramétrer l'invocation. Le résultat de l'invocation devra être également un objet quelconque transmis par copie à l'appelant.
- **nbCalls** permettant de savoir combien de fois le service a été invoqué.
- **getLogs** : renvoyant une liste de logs sur l'utilisation du service. Un log est un ensemble d'information sur une invocation :
 - ◇ la date à laquelle le service a été invoqué
 - ◇ la valeur de l'argument de l'invocation
 - ◇ la valeur du résultat de l'invocation. Si l'invocation a envoyé une erreur, le résultat loggué doit être le message de l'erreur.
- **migrate** : permet de migrer le service vers un hôte passé en paramètre. La migration doit déployer sur l'hôte une copie de l'instance courante au moment de l'appel à **migrate**. Durant le processus de migration, les requêtes sur les autres méthodes de l'instance courante doivent être mises en attente. Une fois la migration terminée, la copie devient l'objet sur lequel les traitements opèrent et l'instance courante se comporte comme un mandataire vis vis de la copie. Les futurs appels à **invoke**, **nbCalls** et **getLogs** sur l'instance courante doivent donc être relayé vers la copie. Si on invoque la méthode **migrate** sur une instance qui a déjà été migrée, alors une exception est jetée.

Un **hôte** est un objet distant qui gère et héberge plusieurs services. Il doit offrir les fonctionnalités suivantes :

- **deployNewService** qui prend un nom de service et une **fabrique de service** en argument. Elle permet d'instancier une nouvelle instance d'un service grâce à la fabrique passée en paramètre et de renvoyer une référence distante sur le service instancié. Si le nom de service existe déjà sur l'hôte, une exception devra être jetée.
- **deployExistingService** qui permet de déployer sur l'hôte un service à partir d'une instance de service déjà existante qui sera passé par copie. La méthode transforme le service en objet distant et renvoie la référence distante ainsi créé.
- **undeployService** qui pour un nom de service donné permet de désactiver le service associé afin qu'il ne soit plus accessible à distance. Si le nom passé en paramètre désigne un service inexistant, la méthode renverra faux, vrai sinon.
- **getRecordedServices** qui renvoie la liste des noms des services existants (déployés ou pas) sur l'hôte.

Une **fabrique de service** est une interface offrant la méthode `Service buildService(String name)`; qui permet d'instancier un nouvel objet de type service avec le nom `name`.

Question 1 7 points

Écrire les interfaces et classes Java RMI permettant de résoudre ce problème. On devra trouver entre autres :

- l'interface **Service** décrivant les méthodes que doit offrir un service
- l'interface **Host** décrivant les méthodes que doit offrir un hôte
- l'interface **ServiceFactory**
- La classe **HostImpl** qui implante l'interface **Host**
- la classe **DivisionService** qui implante l'interface **Service** et **doit respecter toutes les spécifications décrites ci-dessus**. La méthode **invoke** devra calculer la division entière entre deux entiers. Si une des opérandes n'est pas un entier ou bien si la deuxième opérande est égale à 0, une exception doit être jetée. Pour loguer la date de l'invocation vous pourrez utiliser la méthode **System.nanoTime()**.
- la classe **DivisionServiceFactory** qui implante **ServiceFactory**.

Question 2 1 point

Écrire le programme d'un serveur permettant de déployer un hôte et de l'enregistrer sous le nom "*Host*" dans le registry qui est déployé sur la machine "registry.jussieu.fr".

Question 3 1 point

Écrire un programme client qui :

1. déploie sur l'hôte "*Host*" un nouveau **DivisionService** appelé "division"
2. fais une invocation sur le service déployé précédemment pour afficher le résultat de 36 divisé par 4
3. migre le service "division" vers un autre hôte enregistré sous le nom "*Host2*" dans le registry.